Analysis of the Slow Variable X_P

Aim: We have currently got our no drug, TTX, Ouabain and TTX+Ouabain graphs, here we would like to explain why these graphs do what they do

What is it? Activation of the slow potassium current

Why is it important? The time scale for X_P ~8000ms while the next closest variable is ~235ms and a single spike ~5ms. That means that X_P barely moves in the fast timescale a can be treated as a fixed parameter.

In [1]:
```
import sys
from pathlib import Path

REPO_ROOT = Path.cwd().resolve()
if REPO_ROOT.name == 'notebooks':
    REPO_ROOT = REPO_ROOT.parent
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

import numpy as np
import matplotlib.pyplot as plt

from src.plant_kim_fast import (
    STATE_VARS_FAST,
    default_pars_fast,
    build_vode_system_fast,
    simulate,
    equilibrium_curve,
    classify_equilibrium,
    find_folds,
    eq_state_fast,
)

FIGDIR = REPO_ROOT / 'outputs' / 'figures' / 'fast_slow'
FIGDIR.mkdir(parents=True, exist_ok=True)
CACHE = FIGDIR / 'spiking_branch.npz'
NPZ = REPO_ROOT / 'outputs' / 'figures' / 'baseline' / 'baseline_traj.npz

RECOMPUTE = False

# Coarse steps where nothing happens, fine steps where the spiking branch
XP_GRID = np.round(np.concatenate([
    np.arange(0.30, 0.46, 0.02),
    np.arange(0.46, 0.5201, 0.0025),
]), 6)
T_RUN, T_CUT = 12000.0, 4000.0   # ms per frozen X_P; last 8 s analysed
                                 # (spikes can be >1 s apart near the end
SPIKE_THR = -20.0


# ------------------------------------------------------------------------
# 1. Equilibrium branches (closed form, instant)
# ------------------------------------------------------------------------
def equilibrium_branches(xp_lo=0.30, xp_hi=0.60):
    """Split the equilibrium locus into its three branches by stability.
```

```
    rest      stable                          (silent state)
    saddle    unstable, below -30 mV          (firing threshold)
    upper     unstable, above -30 mV          (depolarised equilibrium)
    Points outside a branch are NaN so each plots as its own line.
    """
    V, XP = equilibrium_curve(n=12001)
    inwin = (XP > xp_lo) & (XP < xp_hi)
    stable = np.zeros_like(V, dtype=bool)
    for i in np.where(inwin)[0]:
        stable[i] = classify_equilibrium(V[i], XP[i])['stable']
    masks = {
        'rest': inwin & stable,
        'saddle': inwin & ~stable & (V < -30.0),
        'upper': inwin & ~stable & (V >= -30.0),
    }
    return {k: (np.where(m, XP, np.nan), np.where(m, V, np.nan))
            for k, m in masks.items()}


# ---------------------------------------------------------------------------
# 2. Spiking branch, followed by continuation
#    Each run starts from where the previous one ended, so the simulation
#    stays on the spiking cycle for as long as it exists instead of
#    depending on an arbitrary starting point.
# ---------------------------------------------------------------------------
def follow_spiking_branch(xp_values):
    pars = default_pars_fast()
    ics = eq_state_fast(-45.0)
    out = {'xp': [], 'vmin': [], 'vmax': [], 'isi': [], 'spiking': []}
    for k, xp in enumerate(xp_values):
        p = dict(pars)
        p['X_P'] = float(xp)
        ode = build_vode_system_fast(pars=p, ics=ics, name='PKbranch{}'.f
        pts = simulate(ode, tdata=[0.0, T_RUN], dt=0.25, trajname='br{}'.
        t = np.asarray(pts['t'], dtype=float)
        V = np.asarray(pts['V'], dtype=float)
        ics = {s: float(np.asarray(pts[s], dtype=float)[-1]) for s in STA

        m = t >= T_CUT
        t2, V2 = t[m], V[m]
        cr = np.where((V2[:-1] <= SPIKE_THR) & (V2[1:] > SPIKE_THR))[0]
        isi = float(np.median(np.diff(t2[cr]))) if len(cr) >= 2 else np.n
        spiking = bool(V2.max() > -10.0 and len(cr) >= 2)

        out['xp'].append(float(xp))
        out['vmin'].append(float(V2.min()))
        out['vmax'].append(float(V2.max()))
        out['isi'].append(isi)
        out['spiking'].append(spiking)
        print('  X_P={:.4f}  V in [{:7.2f}, {:7.2f}]  {}'.format(
            xp, V2.min(), V2.max(), 'spiking' if spiking else 'rest'))
    return {k: np.asarray(v) for k, v in out.items()}


if CACHE.exists() and not RECOMPUTE:
    c = np.load(CACHE)
    if np.array_equal(np.round(c['xp'], 6), XP_GRID):
        branch = {k: c[k] for k in c.files}
        print('loaded spiking branch from', CACHE)
```

```
    else:
        RECOMPUTE = True
if not CACHE.exists() or RECOMPUTE:
    print('following the spiking branch ({} runs)...'.format(len(XP_GRID)
    branch = follow_spiking_branch(XP_GRID)
    np.savez(CACHE, **branch)
    print('cached to', CACHE)

eq = equilibrium_branches()
fold = find_folds()[0]
sp = branch['spiking'].astype(bool)
xs = branch['xp'][sp]
xp_end_lo = float(xs.max())
later = branch['xp'][(~sp) & (branch['xp'] > xp_end_lo)]
xp_end_hi = float(later.min()) if len(later) else np.nan
xp_end = xp_end_lo


# ---------------------------------------------------------------------
# 3. One burst cycle of the full 7D trajectory
# ---------------------------------------------------------------------
# If notebook 01 has not been run in this environment, outputs/ will not
# contain the trajectory (outputs/ is usually not committed). Recreate it
# here with exactly the notebook 01 settings; takes about a minute.
if not NPZ.exists():
    from src.plant_kim_model import (
        STATE_VARS, DEFAULT_V0, steady_state_ics, build_vode_system,
    )
    print('baseline_traj.npz not found -- computing the 7D baseline run..
    NPZ.parent.mkdir(parents=True, exist_ok=True)
    ode7 = build_vode_system(ics=steady_state_ics(DEFAULT_V0))
    pts7 = simulate(ode7, tdata=[0.0, 40000.0], dt=0.5, trajname='baselin
    np.savez(NPZ, **{k: np.asarray(pts7[k], dtype=float)
                     for k in ['t', *STATE_VARS]})
    print('saved', NPZ)

d = np.load(NPZ)
t, V, XP = d['t'], d['V'], d['X_P']
m = t >= 10000.0
t, V, XP = t[m], V[m], XP[m]
cr = np.where((V[:-1] <= SPIKE_THR) & (V[1:] > SPIKE_THR))[0]
ts = t[cr]
onsets = [ts[0]] + [ts[i] for i in range(1, len(ts)) if ts[i] - ts[i - 1]
t0, t1 = onsets[0], onsets[1]
last_spike = max(x for x in ts if x < t1)
seg = (t >= t0 - 1500.0) & (t <= t1 + 200.0)
tc, Vc, XPc = t[seg], V[seg], XP[seg]
inb = (t >= t0) & (t <= last_spike)
stages = [                                  # the same four moments as b
    0.5 * (last_spike + t1),                # 1 silent, X_P falling
    t0 - 400.0,                             # 2 burst about to start
    t[inb][np.argmin(V[inb])],              # 3 spiking, X_P rising
    last_spike + 400.0,                     # 4 burst just ended
]


# ---------------------------------------------------------------------
# 4. Figure
# ---------------------------------------------------------------------
RED, BAND = '#C0392B', '0.93'
```

```
lab = dict(fontsize=9, bbox=dict(fc='white', ec='none', alpha=0.85, pad=1

fig, (axA, axB) = plt.subplots(1, 2, figsize=(13, 5.4),
                               gridspec_kw={'width_ratios': [1.0, 1.05]})

# ---- (a) complete bifurcation diagram ---------------------------------
axA.axvspan(fold['X_P'], xp_end, color=BAND, zorder=0)
axA.plot(*eq['upper'], '--', color='0.45', lw=1.3)
axA.plot(*eq['saddle'], 'k--', lw=1.3)
axA.plot(*eq['rest'], 'k-', lw=2.2)
axA.plot(xs, branch['vmax'][sp], '-', color=RED, lw=2.2)
axA.plot(xs, branch['vmin'][sp], '-', color=RED, lw=2.2)
axA.plot(fold['X_P'], fold['V'], 'o', ms=8, mfc='white', mec='k', mew=1.5
for v in (branch['vmax'][sp][-1], branch['vmin'][sp][-1]):
    axA.plot(xp_end, v, 'x', ms=8, mew=2, color=RED, zorder=5)

axA.text(0.305, 22.0, 'spiking cycle (max)', color=RED, **lab)
axA.text(0.305, -44.5, 'spiking cycle (min)', color=RED, **lab)
axA.text(0.305, -12.0, 'unstable equilibrium', color='0.35', **lab)
axA.text(0.595, -36.5, 'threshold (saddle)', ha='right', **lab)
axA.text(0.595, -58.5, 'rest (stable)', ha='right', va='bottom', **lab)
axA.text(0.5 * (fold['X_P'] + xp_end), 4.0, 'rest and spiking\nboth possi
         ha='center', va='center', fontsize=9, color='0.35')
axA.annotate('fold', (fold['X_P'], fold['V']), xytext=(-38, 22),
             textcoords='offset points', fontsize=9,
             arrowprops=dict(arrowstyle='-', color='k', lw=0.8))
axA.annotate('spiking\nends', (xp_end, branch['vmin'][sp][-1]), xytext=(1
             textcoords='offset points', fontsize=9, color=RED,
             arrowprops=dict(arrowstyle='-', color=RED, lw=0.8))

axA.set_xlim(0.30, 0.60)
axA.set_ylim(-60, 33)
axA.set_xlabel(r'$X_P$  (held fixed)')
axA.set_ylabel('V (mV)')
axA.set_title('(a)  What the fast subsystem can do at each fixed $X_P$',
              fontsize=11, loc='left')
axA.grid(True, alpha=0.2)

# ---- (b) one burst cycle, zoomed --------------------------------------
axB.axvspan(fold['X_P'], xp_end, color=BAND, zorder=0)
axB.plot(XPc, Vc, color='0.6', lw=0.9, zorder=2)
axB.plot(*eq['saddle'], 'k--', lw=1.3, zorder=3)
axB.plot(*eq['rest'], 'k-', lw=2.2, zorder=3)
axB.plot(xs, branch['vmin'][sp], '-', color=RED, lw=2.2, zorder=3)
axB.plot(fold['X_P'], fold['V'], 'o', ms=8, mfc='white', mec='k', mew=1.5
axB.plot(xp_end, branch['vmin'][sp][-1], 'x', ms=8, mew=2, color=RED, zor

for n, tt in enumerate(stages, start=1):
    xi, Vi = np.interp(tt, tc, XPc), np.interp(tt, tc, Vc)
    axB.plot(xi, Vi, 'o', ms=15, mfc='white', mec='0.2', mew=1.2, zorder=
    axB.text(xi, Vi, str(n), ha='center', va='center', fontsize=9.5, zord

axB.text(0.553, -53.9, 'rest', ha='right', va='bottom', **lab)
axB.text(0.553, -43.6, 'threshold', ha='right', va='top', **lab)
axB.text(0.335, -49.2, 'spiking (lowest V)', color=RED, **lab)
axB.text(0.335, -54.1, 'grey: real burst (7D model)', color='0.4', va='bo

axB.set_xlim(0.33, 0.56)
axB.set_ylim(-54.5, -43.0)
axB.set_xlabel(r'$X_P$')
```

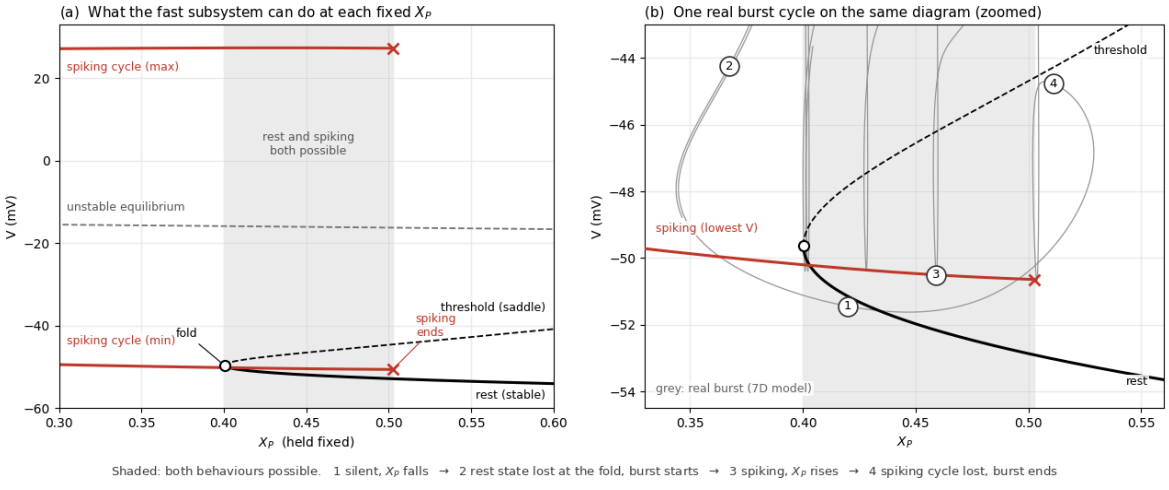
```
axB.set_ylabel('V (mV)')
axB.set_title('(b)  One real burst cycle on the same diagram (zoomed)',
              fontsize=11, loc='left')
axB.grid(True, alpha=0.2)

fig.text(0.5, 0.015,
         r'Shaded: both behaviours possible.   '
         r'1 silent, $X_P$ falls  $\rightarrow$  2 rest state lost at the
         r'$\rightarrow$  3 spiking, $X_P$ rises  $\rightarrow$  4 spikin
         ha='center', fontsize=9.5, color='0.25')
fig.subplots_adjust(left=0.06, right=0.985, top=0.92, bottom=0.15, wspace

out = FIGDIR / 'fast_slow_figure.png'
fig.savefig(out, dpi=200)
plt.show()
print('saved', out)


# ------------------------------------------------------------------------
# 5. Numbers to quote
# ------------------------------------------------------------------------
print('\nfold (burst starts):        X_P = {:.4f},  V = {:.2f} mV'.format
if np.isfinite(xp_end_hi):
    print('spiking ends (burst ends):  between X_P = {:.4f} and {:.4f}'.f
else:
    print('spiking ends (burst ends):  beyond X_P = {:.4f} (extend XP_GRI
print('interspike interval approaching the end:')
for x, isi in zip(branch['xp'][sp][-6:], branch['isi'][sp][-6:]):
    print('   X_P = {:.4f}   ISI = {}'.format(x, 'n/a' if not np.isfinite
```

```
loaded spiking branch from /outputs/figures/fast_slow/spiking_branch.npz
```




```
saved /outputs/figures/fast_slow/fast_slow_figure.png

fold (burst starts):        X_P = 0.4003,  V = -49.63 mV
spiking ends (burst ends):  between X_P = 0.5025 and 0.5050
interspike interval approaching the end:
   X_P = 0.4900   ISI = 1209 ms
   X_P = 0.4925   ISI = 1307 ms
   X_P = 0.4950   ISI = 1432 ms
   X_P = 0.4975   ISI = 1605 ms
   X_P = 0.5000   ISI = 1874 ms
   X_P = 0.5025   ISI = 2458 ms
```

Left: Panel shows just the bifurcation diagram. Red lines are the minimum and maximum of spiking cycles. Past this point on the right no limit cycles and occur and

so you can't get any spiking. The black stable equilibria on the bottom is the silent phase of the system (the hyperpolarizing part). Right: Now we've overlapped the 7D trajectory under the bifurcation graph. Spikes start and end with the red limit cycles and bursting starts when we reach the fold bifurcation and ends when we read the end of the red branch

Now we can freeze the values of X_P and plot the trajectories and plots of V vs X_K to show the limit cycles

In [5]:
```
import sys
from pathlib import Path

REPO_ROOT = Path.cwd().resolve()
if REPO_ROOT.name == 'notebooks':
    REPO_ROOT = REPO_ROOT.parent
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

import numpy as np
import matplotlib.pyplot as plt

from src.plant_kim_fast import (
    STATE_VARS_FAST,
    default_pars_fast,
    build_vode_system_fast,
    simulate,
    fast_equilibria,
    eq_state_fast,
    classify_equilibrium,
)

FIGDIR = REPO_ROOT / 'outputs' / 'figures' / 'fast_slow'
FIGDIR.mkdir(parents=True, exist_ok=True)

XP_VALUES = [0.35, 0.44, 0.50, 0.52]
KICK = 1.0        # mV above / below the threshold to start from
T_END = 8000.0    # ms per run
T_LATE = 4000.0   # ms of tail used for the phase portrait
DT = 0.25
SPIKE_THR = -20.0

RED, BLUE = '#C0392B', '#1F5C99'


def run_fast(X_P, ics, name):
    """Integrate the frozen-X_P fast subsystem from one initial condition
    pars = default_pars_fast()
    pars['X_P'] = float(X_P)
    ode = build_vode_system_fast(pars=pars, ics=ics, name=name)
    pts = simulate(ode, tdata=[0.0, T_END], dt=DT, trajname=name)
    return {k: np.asarray(pts[k], dtype=float) for k in ['t', *STATE_VARS


def starting_points(X_P):
    """Two ICs per X_P: just above and just below the threshold if there
    one, otherwise a depolarised and a hyperpolarised start."""
    roots = fast_equilibria(X_P)
```

```
    if len(roots) >= 2:
        V_saddle = roots[1]
        base = eq_state_fast(V_saddle)
        out = []
        for dV, tag in [(+KICK, 'above threshold'), (-KICK, 'below thresh
            ics = dict(base)
            ics['V'] = V_saddle + dV
            out.append((ics, tag))
        return out, V_saddle
    return [(eq_state_fast(-42.0), 'from -42 mV'),
            (eq_state_fast(-52.0), 'from -52 mV')], None


fig, axes = plt.subplots(2, len(XP_VALUES), figsize=(4.0 * len(XP_VALUES)
                         gridspec_kw={'height_ratios': [1.0, 1.25]})
summary = []

for col, X_P in enumerate(XP_VALUES):
    axT, axB = axes[0, col], axes[1, col]
    ics_list, V_saddle = starting_points(X_P)
    roots = fast_equilibria(X_P)

    for j, (ics, tag) in enumerate(ics_list):
        d = run_fast(X_P, ics, 'PKslice{}_{}'.format(col, j))
        t, V, X_K = d['t'], d['V'], d['X_K']
        late = t >= (T_END - T_LATE)
        spiking = bool(V[late].max() > -10.0)
        colour = RED if spiking else BLUE

        axT.plot(t / 1000.0, V, lw=0.9, color=colour, label=tag)
        axB.plot(X_K[late], V[late], lw=1.0, color=colour)

        cr = np.where((V[late][:-1] <= SPIKE_THR) & (V[late][1:] > SPIKE_
        isi = float(np.median(np.diff(t[late][cr]))) if len(cr) >= 2 else
        summary.append((X_P, tag, 'spiking' if spiking else 'rest',
                        float(V[late].min()), float(V[late].max()), isi))

    # equilibria of this slice
    for V_eq in roots:
        if V_eq < -30.0:
            st = classify_equilibrium(V_eq, X_P)['stable']
            s = eq_state_fast(V_eq)
            axB.plot(s['X_K'], V_eq, 'o', ms=8, zorder=6,
                     mfc=('k' if st else 'white'), mec='k', mew=1.5)
    if V_saddle is not None:
        axT.axhline(V_saddle, color='0.5', ls='--', lw=0.9)

    kinds = {r for _, _, r, *_ in summary[-len(ics_list):]}
    verdict = ('spiking only' if kinds == {'spiking'} else
               'rest only' if kinds == {'rest'} else 'BOTH possible')
    axT.set_title('$X_P$ = {:.2f}\n{}'.format(X_P, verdict), fontsize=11)
    axT.set_xlabel('t (s)')
    axT.set_ylim(-58, 33)
    axT.grid(True, alpha=0.25)
    axT.legend(fontsize=8, loc='upper right', framealpha=0.9)

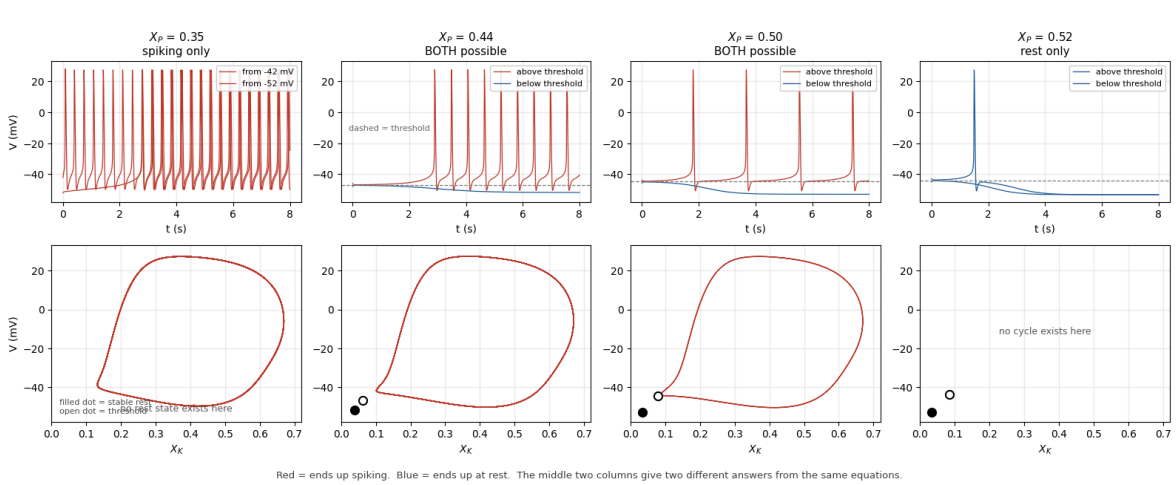
    axB.set_xlabel('$X_K$')
    axB.set_xlim(0.0, 0.72)
    axB.set_ylim(-58, 33)
    axB.grid(True, alpha=0.25)
```

```
    if col == 0:
        axT.set_ylabel('V (mV)')
        axB.set_ylabel('V (mV)')
        axB.text(0.03, 0.04, 'filled dot = stable rest\nopen dot = thresh
                 transform=axB.transAxes, fontsize=8, va='bottom', color=

axes[0, 1].text(0.03, 0.55, 'dashed = threshold', transform=axes[0, 1].tr
                va='top', fontsize=8, color='0.4')
axes[1, 0].text(0.5, 0.06, 'no rest state exists here', transform=axes[1,
                ha='center', fontsize=9, color='0.35')
axes[1, -1].text(0.5, 0.5, 'no cycle exists here', transform=axes[1, -1].
                 ha='center', fontsize=9, color='0.35')
fig.suptitle('The fast subsystem at four frozen values of $X_P$   '
             '(top: V against time.  bottom: the attractor in the $(X_K,
             fontsize=11)
fig.tight_layout(rect=[0, 0.03, 1, 0.94])
fig.text(0.5, 0.008,
         'Red = ends up spiking.  Blue = ends up at rest.  '
         'The middle two columns give two different answers from the same
         ha='center', fontsize=9.5, color='0.25')

out = FIGDIR / 'frozen_slices.png'
fig.savefig(out, dpi=200)
plt.show()
print('saved', out)

print('\n{:>6}  {:<16} {:<8} {:>8} {:>8} {:>10}'.format(
    'X_P', 'start', 'ends as', 'V_min', 'V_max', 'ISI'))
for X_P, tag, regime, vmn, vmx, isi in summary:
    print('{:6.2f}  {:<16} {:<8} {:8.2f} {:8.2f} {:>10}'.format(
        X_P, tag, regime, vmn, vmx,
        'n/a' if not np.isfinite(isi) else '{:.0f} ms'.format(isi)))
```



saved /outputs/figures/fast_slow/frozen_slices.png

X_P	start	ends as	V_min	V_max	ISI
0.35	from −42 mV	spiking	−49.87	27.30	342 ms
0.35	from −52 mV	spiking	−49.87	27.30	342 ms
0.44	above threshold	spiking	−50.42	27.40	581 ms
0.44	below threshold	rest	−51.73	−50.63	n/a
0.50	above threshold	spiking	−50.64	27.31	1874 ms
0.50	below threshold	rest	−52.87	−52.60	n/a
0.52	above threshold	rest	−53.15	−52.28	n/a
0.52	below threshold	rest	−53.15	−52.98	n/a

We can see that past a certain values of X_P we can get what we observe from the initial bifurcation diagram which is the regions where only spiking is possible, where both are possible and where no spiking can occur. Boundaries are 0.400 and 0.503. We also notice there is 'bistability' meaning there are two stable states that are possible.

Bifurcation diagrams of X_P for the drugs under 1. no drug 2. TTX 3. Ouabain 4. TTX + Ouabain

In [3]:
```
import sys
from pathlib import Path

REPO_ROOT = Path.cwd().resolve()
if REPO_ROOT.name == 'notebooks':
    REPO_ROOT = REPO_ROOT.parent
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

from src.plant_kim_fast import (
    STATE_VARS_FAST,
    default_pars_fast,
    build_vode_system_fast,
    simulate,
    equilibrium_curve,
    classify_equilibrium,
    find_folds,
    eq_state_fast,
)
from src.plant_kim_model import (
    BASELINE_PARS,
    STATE_VARS,
    DEFAULT_V0,
    steady_state_ics,
    build_vode_system,
)

FIGDIR = REPO_ROOT / 'outputs' / 'figures' / 'fast_slow'
CACHE = FIGDIR / 'drug_cache'
CACHE.mkdir(parents=True, exist_ok=True)

RECOMPUTE = False
T_RUN, T_LATE = 10000.0, 6000.0     # ms per frozen-X_P run, and tail ana
DT = 0.25
SPIKE_THR = -20.0
RED, GREY, BAND = '#C0392B', '0.55', '#D6E4F0'

CONDITIONS = [
    dict(key='untreated', title='Untreated', drug={},
         grid=np.arange(0.30, 0.541, 0.02), xlim=(0.30, 0.60), ylim=(-58,
         t_end=40000.0, t_cut=10000.0),
    dict(key='ttx', title=r'TTX  ($g_I = 0$)', drug={'g_I': 0.0},
         grid=np.arange(0.30, 0.641, 0.03), xlim=(0.30, 0.64), ylim=(-56,
```

```
         t_end=80000.0, t_cut=20000.0),
    dict(key='ouabain', title=r'Ouabain  ($I_{ep} = 0$)', drug={'I_ep': 0
         grid=np.arange(0.80, 1.001, 0.02), xlim=(0.78, 1.00), ylim=(-58,
         t_end=60000.0, t_cut=15000.0),
    dict(key='both', title=r'TTX + ouabain', drug={'g_I': 0.0, 'I_ep': 0.
         grid=np.arange(0.90, 1.001, 0.01), xlim=(0.90, 1.00), ylim=(-56,
         t_end=120000.0, t_cut=40000.0),
]


# ------------------------------------------------------------------------
# helpers
# ------------------------------------------------------------------------
def fast_pars(drug):
    p = default_pars_fast()
    p.update({k: float(v) for k, v in drug.items()})
    return p


def spiking_branch(drug, grid):
    """Follow the fast subsystem's attractor across frozen X_P values."""
    base = fast_pars(drug)
    ics = eq_state_fast(-45.0)
    rows = {'xp': [], 'vmin': [], 'vmax': [], 'spiking': []}
    for k, xp in enumerate(grid):
        p = dict(base)
        p['X_P'] = float(xp)
        ode = build_vode_system_fast(pars=p, ics=ics, name='PKdrug{}'.for
        pts = simulate(ode, tdata=[0.0, T_RUN], dt=DT, trajname='dg{}'.fo
        t = np.asarray(pts['t'], dtype=float)
        V = np.asarray(pts['V'], dtype=float)
        ics = {s: float(np.asarray(pts[s], dtype=float)[-1]) for s in STA
        m = t >= (T_RUN - T_LATE)
        V2 = V[m]
        cr = np.where((V2[:-1] <= SPIKE_THR) & (V2[1:] > SPIKE_THR))[0]
        rows['xp'].append(float(xp))
        rows['vmin'].append(float(V2.min()))
        rows['vmax'].append(float(V2.max()))
        rows['spiking'].append(bool(V2.max() > -10.0 and len(cr) >= 2))
    return {k: np.asarray(v) for k, v in rows.items()}


def full_trajectory(drug, t_end):
    """7D run for this condition."""
    p = dict(BASELINE_PARS)
    p.update({k: float(v) for k, v in drug.items()})
    ode = build_vode_system(pars=p, ics=steady_state_ics(DEFAULT_V0))
    pts = simulate(ode, tdata=[0.0, t_end], dt=0.5, trajname='drugtraj')
    return {k: np.asarray(pts[k], dtype=float) for k in ['t', *STATE_VARS


def load_or_build(cond):
    f = CACHE / '{}.npz'.format(cond['key'])
    if f.exists() and not RECOMPUTE:
        c = np.load(f)
        return {k: c[k] for k in c.files}
    print('computing {} ...'.format(cond['key']), flush=True)
    br = spiking_branch(cond['drug'], cond['grid'])
    tr = full_trajectory(cond['drug'], cond['t_end'])
    data = {'br_' + k: v for k, v in br.items()}
```

```
    data.update({'tr_' + k: tr[k] for k in ['t', 'V', 'X_P']})
    np.savez(f, **data)
    print('  cached to', f)
    return data


def regime_of(t, V, t_cut):
    m = t >= t_cut
    t2, V2 = t[m], V[m]
    cr = np.where((V2[:-1] <= SPIKE_THR) & (V2[1:] > SPIKE_THR))[0]
    if V2.max() - V2.min() < 0.5:
        return 'rest'
    if len(cr) < 2:
        return 'slow wave, no spikes'
    gaps = np.diff(t2[cr])
    return 'bursting' if (gaps > 1500.0).any() else 'continuous beating'


# ------------------------------------------------------------------------
# figure
# ------------------------------------------------------------------------
fig, axes = plt.subplots(2, 2, figsize=(12.5, 8.6))
summary = []

for ax, cond in zip(axes.ravel(), CONDITIONS):
    d = load_or_build(cond)
    pars = fast_pars(cond['drug'])
    lo, hi = cond['xlim']

    t, V, XP = d['tr_t'], d['tr_V'], d['tr_X_P']
    m = t >= cond['t_cut']
    xp_lo, xp_hi = float(XP[m].min()), float(XP[m].max())
    ax.axvspan(xp_lo, xp_hi, color=BAND, zorder=0)

    Vc, XPc = equilibrium_curve(pars, n=12001)
    inwin = (XPc > lo - 0.02) & (XPc < hi + 0.02)
    stable = np.zeros_like(Vc, dtype=bool)
    for i in np.where(inwin)[0]:
        stable[i] = classify_equilibrium(Vc[i], XPc[i], pars)['stable']
    ax.plot(np.where(inwin & ~stable, XPc, np.nan),
            np.where(inwin & ~stable, Vc, np.nan), 'k--', lw=1.2, zorder=
    ax.plot(np.where(inwin & stable, XPc, np.nan),
            np.where(inwin & stable, Vc, np.nan), 'k-', lw=2.2, zorder=3)

    sp = d['br_spiking'].astype(bool)
    if sp.any():
        ax.plot(d['br_xp'][sp], d['br_vmax'][sp], '-', color=RED, lw=2.2,
        ax.plot(d['br_xp'][sp], d['br_vmin'][sp], '-', color=RED, lw=2.2,
    else:
        ax.text(0.5, 0.88, 'no spiking cycle at any $X_P$', transform=ax.
                ha='center', fontsize=9.5, color=RED)

    ax.plot(XP[m], V[m], lw=0.7, color=GREY, zorder=2)

    folds = find_folds(pars, XP_window=(lo - 0.05, hi + 0.05))
    for f in folds:
        ax.plot(f['X_P'], f['V'], 'o', ms=8, mfc='white', mec='k', mew=1.

    reg = regime_of(t, V, cond['t_cut'])
    ax.set_title('{}   -->   {}'.format(cond['title'], reg), fontsize=11)
```

```
    ax.set_xlim(*cond['xlim'])
    ax.set_ylim(*cond['ylim'])
    ax.set_xlabel(r'$X_P$')
    ax.set_ylabel('V (mV)')
    ax.grid(True, alpha=0.2)

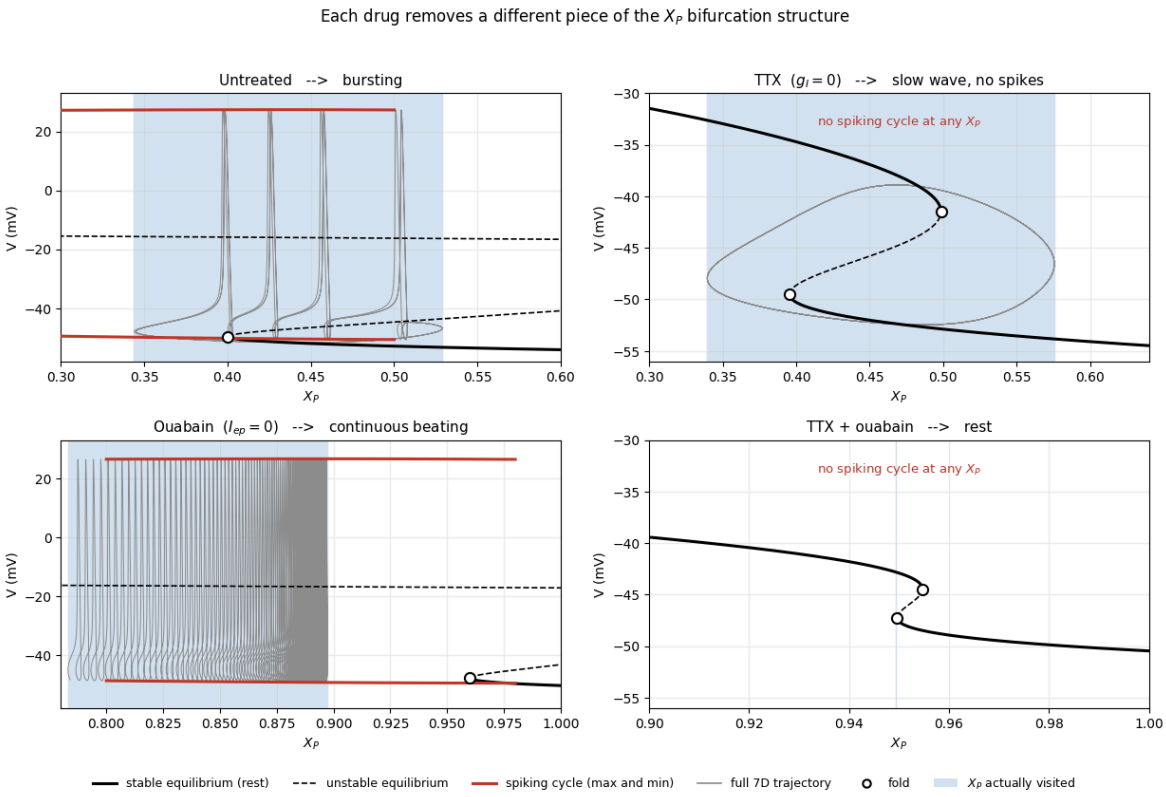
    summary.append((cond['title'], reg, [round(f['X_P'], 4) for f in fold
                    (round(xp_lo, 3), round(xp_hi, 3)),
                    (round(float(d['br_xp'][sp].min()), 3),
                     round(float(d['br_xp'][sp].max()), 3)) if sp.any() e

handles = [
    Line2D([], [], color='k', lw=2.2, label='stable equilibrium (rest)'),
    Line2D([], [], color='k', lw=1.2, ls='--', label='unstable equilibriu
    Line2D([], [], color=RED, lw=2.2, label='spiking cycle (max and min)'
    Line2D([], [], color=GREY, lw=1.2, label='full 7D trajectory'),
    Line2D([], [], color='k', marker='o', ls='none', mfc='white', mew=1.5
    Patch(facecolor=BAND, label='$X_P$ actually visited'),
]
fig.legend(handles=handles, loc='lower center', ncol=6, fontsize=9,
           frameon=False, bbox_to_anchor=(0.5, 0.005))
fig.suptitle('Each drug removes a different piece of the $X_P$ bifurcatio
             fontsize=12)
fig.tight_layout(rect=[0, 0.05, 1, 0.96])

out = FIGDIR / 'drug_xp_diagrams.png'
fig.savefig(out, dpi=200)
plt.show()
print('saved', out)

print('\n{:<22} {:<22} {:<22} {:<18} {}'.format(
    'condition', 'regime', 'fold(s) at X_P', 'X_P visited', 'spiking bran
for title, reg, folds, visited, br in summary:
    print('{:<22} {:<22} {:<22} {:<18} {}'.format(
        title.replace('$', '').replace('\\', ''), reg, str(folds), str(vi
        'none' if br is None else '{} to {}'.format(*br)))
```

```
computing ttx ...
  cached to /outputs/figures/fast_slow/drug_cache/ttx.npz
computing ouabain ...
IOStream.flush timed out
IOStream.flush timed out
IOStream.flush timed out
IOStream.flush timed out
  cached to /outputs/figures/fast_slow/drug_cache/ouabain.npz
computing both ...
IOStream.flush timed out
IOStream.flush timed out
IOStream.flush timed out
IOStream.flush timed out
IOStream.flush timed out
  cached to /outputs/figures/fast_slow/drug_cache/both.npz
```




```
saved /outputs/figures/fast_slow/drug_xp_diagrams.png
```

```
condition              regime                fold(s) at X_P         X_P v
isited        spiking branch
Untreated              bursting              [0.4003]               (0.34
4, 0.529)     0.3 to 0.5
TTX  (g_I = 0)         slow wave, no spikes  [0.3955, 0.4985]       (0.3
4, 0.576)      none
Ouabain  (I_{ep} = 0)  continuous beating    [0.9601]               (0.78
3, 0.897)     0.8 to 0.98
TTX + ouabain          rest                  [0.9496, 0.9547]       (0.94
9, 0.949)     none
```

Summary: Ultimately, the bursting can be explained as the spiking pushes X_P up until it falls onto the stable equilibria where the silent branch pulls X_P down until the fold bifurcation. We also get that our bursts start and stop at 0.400 and 0.503 which is calculated without the simulating the burst. Ouabain pushes the fold up 0.96 where the cell only goes to 0.89 so the resting state becomes unreachable so the cell continuously spikes. TTX gets rid of the spiking branch entirely so the cell can never spike